

Final Project — Developing a Dynamic Website with Laravel

1. Learning Objective

This project aims to enable students to design and develop a complete dynamic web application using the Laravel framework, by mobilizing all the technical and conceptual skills acquired in the four practical sessions (TP1 to TP4). The topic is defined based on the context of the academic track (see section 2 below) and must be sufficiently rich to cover CRUD functionalities, MVC organization, data management with Eloquent, use of Blade views, validation, pagination, etc.

2. Project Context

- Students of 2-LBC-BIS: the topic is the one selected as part of the module “Professional Supervised Project 2”.
- Students of 2-LBC-BI: the topic is the one developed in the “Software Engineering” module.

 **Important Note:** the chosen topic must necessarily allow:

- The implementation of several entities with relationships between them.
- The implementation of at least as many CRUD functionalities as there are members in the group.
- Each student must be responsible for at least one complete CRUD functionality.

Expected Deliverables

Deliverable	Description
Complete source code	Compressed Laravel project (.zip) or hosted on GitHub (with README)
Submission deadline in the Blackboard submission area will be the day before the presentation.	

3. Targeted Skills

Upon completion of this project, the student will be able to:

- Design and model a relational database consistent with the business domain.
- Define and organize routes, controllers, and Blade views in an MVC architecture.
- Manipulate data using Laravel’s Eloquent ORM.
- Create responsive and user-friendly interfaces.

- Implement server-side validated forms.
- Manage pagination and navigation between views.
- Add user authentication with role-based access control.

4. Project Implementation Structure

✓ Step 1: Laravel Project Setup (TP1 & TP2)

- Create the project with `laravel new project_name`.
- Define the base structure: main layout, folder organization, asset inclusion.
- Create static pages: home, about, contact.

✓ Step 2: View Management with HTML and Blade (TP2 & TP3)

- Use directives `@extends`, `@section`, `@yield`, `@include` for view composition.
- Create reusable Blade Components (e.g., card, alert, form, button).
- Develop CRUD interface pages.

✓ Step 3: Controllers, Routes and Forms (TP2, TP3, TP4)

- Generate controllers using `php artisan make:controller`.
- Implement CRUD methods: `index`, `create`, `store`, `edit`, `update`, `destroy`, `show`.
- Set up secure forms with CSRF, `@csrf`, `method_field`, etc.
- Validate data with `request->validate()`.

✓ Step 4: Database and Models (TP4)

- Create migrations with appropriate fields (types, constraints, relationships).
- Insert mock data via forms.
- Define Eloquent models and relationships: `hasOne`, `hasMany`, `belongsTo`.

✓ Step 5: Navigation, Search and Pagination (TP4)

- Set up navigation between views (e.g., "Back", "View details" buttons).
- Add a search field to filter results.
- Implement pagination using `paginate()` and `links()` in Blade.

5. Rules and Recommendations

- Compliance with the MVC (Model-View-Controller) pattern is required.
- Each group member must present their CRUD functionality during the oral presentation.
- The project must include at least:
 - Entities with relationships.
 - A homepage and a contact page.
 - Forms with validation.
 - CRUD operations.
- Bonus: authentication, pagination and search, file upload.